

JAVA WITH DSA-DAY 9

LINKED LIST

```
class Node {  
    int data;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}  
  
public class LinkedList {  
    static Node head = null;  
    // Insert at Beginning  
    static void insertFirst(int data) {  
        Node newNode = new Node(data);  
        newNode.next = head;  
        head = newNode;  
    }  
    // Insert at End  
    static void insertLast(int data) {  
        Node newNode = new Node(data);  
        if (head == null) {  
            head = newNode;  
            return;  
        }  
        Node temp = head;  
        while (temp.next != null) {  
            temp = temp.next;  
        }  
        temp.next = newNode;  
    }  
}
```

// Insert at Position (1-based)

```
static void insertPosition(int pos, int data) {  
    if (pos <= 0) {  
        System.out.println("Invalid Position");  
        return;  
    }  
    if (pos == 1) {  
        insertFirst(data);  
        return;  
    }
```

```
    Node newNode = new Node(data);  
    Node temp = head;  
    for (int i = 1; i < pos - 1; i++) {  
        if (temp == null) {  
            System.out.println("Invalid Position");  
            return;  
        }  
        temp = temp.next;  
    }  
    if (temp == null) {  
        System.out.println("Invalid Position");  
        return;  
    }  
    newNode.next = temp.next;  
    temp.next = newNode;  
}
```

// Delete First

```
static void deleteFirst() {  
    if (head == null) {  
        System.out.println("List is Empty");
```

```
        return;
    }
    head = head.next;
}
```

// Delete Last

```
static void deleteLast() {
    if (head == null) {
        System.out.println("List is Empty");
        return;
    }
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next.next != null) {
        temp = temp.next;
    }
    temp.next = null;
}
```

// Delete at Position (1-based)

```
static void deletePosition(int pos) {
    if (head == null) {
        System.out.println("List is Empty");
        return;
    }
    if (pos <= 0) {
        System.out.println("Invalid Position");
        return;
    }
    if (pos == 1) {
```

```

        head = head.next;
        return;
    }
    Node temp = head;
    for (int i = 1; i < pos - 1; i++) {
        if (temp == null || temp.next == null) {
            System.out.println("Invalid Position");
            return;
        }
        temp = temp.next;
    }
    if (temp.next == null) {
        System.out.println("Invalid Position");
        return;
    }
    temp.next = temp.next.next;
}

```

// Display

```

static void display() {
    if (head == null) {
        System.out.println("List is Empty");
        return;
    }
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
    System.out.println("null");
}

public static void main(String[] args) {
    System.out.println("Insert Operations");
}

```

```

insertFirst(20);
insertFirst(10);
insertLast(30);
insertPosition(3, 25);
display();
System.out.println("\nDelete First");
deleteFirst();
display();
System.out.println("\nDelete Last");
deleteLast();
display();
System.out.println("\nDelete Position");
deletePosition(2);
display();
}
}

```

OUTPUT:

Insert Operations

10 -> 20 -> 25 -> 30 -> null

Delete First

20 -> 25 -> 30 -> null

Delete Last

20 -> 25 -> null

Delete Position

20 -> null

VECTOR

```

import java.util.Vector;

public class Main {

    public static void main(String[] args) {

        // Create Vector
    }
}

```

```
Vector<Integer> v = new Vector<>();  
  
// Add elements  
v.add(10);  
v.add(20);  
v.add(30);  
v.add(40);  
System.out.println("After add: " + v);  
  
// Add element at index  
v.add(2, 25);  
System.out.println("After add at index 2: " + v);  
  
// Access element  
System.out.println("Element at index 3: " + v.get(3));  
  
// Update element  
v.set(1, 100);  
System.out.println("After set: " + v);  
  
// Remove by index  
v.remove(2);  
System.out.println("After remove index 2: " + v);  
  
// Remove by value  
v.remove(Integer.valueOf(30));  
System.out.println("After remove value 30: " + v);  
  
// Search  
System.out.println("Contains 40? " + v.contains(40));  
  
// Index  
System.out.println("Index of 40: " + v.indexOf(40));
```

```
// Size
System.out.println("Size: " + v.size());

// Capacity
System.out.println("Capacity: " + v.capacity());

// First and Last
System.out.println("First Element: " + v.firstElement());
System.out.println("Last Element: " + v.lastElement());

// Display using normal for loop
System.out.print("Vector Elements: ");
for (int i = 0; i < v.size(); i++) {
    System.out.print(v.get(i) + " ");
}
System.out.println();

// Empty check
System.out.println("Is Empty? " + v.isEmpty());

// Clear
v.clear();
System.out.println("After clear: " + v);

// Empty check again
System.out.println("Is Empty now? " + v.isEmpty());
}
}
```

OUTPUT:

After add: [10, 20, 30, 40]

After add at index 2: [10, 20, 25, 30, 40]

Element at index 3: 30

After set: [10, 100, 25, 30, 40]

After remove index 2: [10, 100, 30, 40]

After remove value 30: [10, 100, 40]

Contains 40? true

Index of 40: 2

Size: 3

Capacity: 10

First Element: 10

Last Element: 40

Vector Elements: 10 100 40

Is Empty? false

After clear: []

Is Empty now? true

STACK

```
import java.util.Stack;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Stack<Integer> stack = new Stack<>();
```

```
        // Push
```

```
        stack.push(10);
```

```
        stack.push(20);
```

```
        stack.push(30);
```

```
        stack.push(40);
```

```
        System.out.println("Stack after push: " + stack);
```

```
        // Peek
```

```
        System.out.println("Top element: " + stack.peek());
```

```
        // Pop
```

```
        int removed = stack.pop();
```

```
        System.out.println("Popped element: " + removed);
```

```
        System.out.println("Stack after pop: " + stack);
```

```
        // Search
```

```
int pos = stack.search(20);
if (pos != -1)
    System.out.println("20 found at position from top: "+pos);
else
    System.out.println("20 not found");

// Size
System.out.println("Stack size: " + stack.size());

// isEmpty
System.out.println("Is stack empty? " + stack.isEmpty());

// Display
System.out.println("Elements in stack:");
for (int i = 0; i < stack.size(); i++) {
    System.out.print(stack.get(i) + " ");
}
System.out.println();

// Clear
stack.clear();
System.out.println("After clear: " + stack);
System.out.println("Is stack empty now? " + stack.isEmpty());
}
}
```

OUTPUT:

Stack after push: [10, 20, 30, 40]

Top element: 40

Popped element: 40

Stack after pop: [10, 20, 30]

20 found at position from top: 2

Stack size: 3

Is stack empty? false

Elements in stack:

10 20 30

After clear: []

Is stack empty now? True

VALID PARANTHESIS

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Parentheses: ");

        String s = sc.nextLine();

        Stack<Character> stack = new Stack<>();

        boolean valid = true;

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            // Opening brackets

            if (ch == '(' || ch == '{' || ch == '[') {

                stack.push(ch);

            }

            // Closing brackets

            else {

                if (stack.isEmpty()) {

                    valid = false;

                    break;

                }

                char top = stack.pop();

                if ((ch == ')' && top != '(') ||

                    (ch == '}' && top != '{') ||

                    (ch == ']' && top != '[')) {

                    valid = false;

                    break;

                }

            }

        }

    }

}
```

```

        }
    }
    if (!stack.isEmpty()) {
        valid = false;
    }
    if (valid)
        System.out.println("Valid Parenthesis");
    else
        System.out.println("Invalid Parenthesis");
}
}

```

OUTPUT 1:

Enter Parentheses: [()]{}

Valid Parenthesis

OUTPUT 2:

Enter Parentheses: [()]{}

Invalid Parenthesis

REMOVE ALL DIGITS FROM A GIVEN STRING AND DISPLAY ONLY THE CHARACTERS

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter String: ");

        String s = sc.nextLine();

        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {

            char ch=s.charAt(i);

            if(!Character.isDigit(ch)){

                stack.push(ch);

            }

        }

        String result="";

        for(int i=0;i<stack.size();i++){

```

```

        result+=stack.get(i);
    }
    System.out.println(""+result);
}
}

```

OUTPUT 1:

Enter String: He3l2lo

Hello

OUTPUT 2:

Enter String: Anuk901294e78e912rtha212na

Anukeerthana

REMOVE CONSECUTIVE DUPLICATE CHARACTERS FROM A STRING AND PRINT THE REMAINING CHARACTERS

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter String: ");

        String s = sc.nextLine();

        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {

            char ch=s.charAt(i);

            if(!stack.isEmpty() && stack.peek() == ch){

                stack.pop();

            }

            else{

                stack.push(ch);

            }

        }

        String result="";

        for(int i=0;i<stack.size();i++){

            result+=stack.get(i);

        }

    }

}

```

```
        System.out.println(""+result);
    }
}
```

OUTPUT 1:

Enter String: mississippi
m

OUTPUT 2:

Enter String: pqqprs
rs

OUTPUT:

Enter String: PQPQRS
PQPQRS

PALINDROME USING STACK

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter String: ");

        String s = sc.nextLine();

        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {

            char ch=s.charAt(i);

            stack.push(ch);

        }

        boolean palindrome=true;

        for(int i=0;i<stack.size();i++){

            if(s.charAt(i)!=stack.pop()){

                palindrome=false;

                break;

            }

        }

        if(palindrome){

            System.out.println("it is a palindrome");

        }

    }

}
```

```

    }
    else{
        System.out.println("it is not a palindrome");
    }
}
}

```

OUTPUT 1:

Enter String: gfdgdgf

it is not a palindrome

OUTPUT 2:

Enter String: MADAM

it is a palindrome

POSTFIX

2 3 * 4 5 * + 9 - =17

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Postfix Expression: ");
        String s = sc.nextLine();
        Stack<Integer> stack = new Stack<>();
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isDigit(ch)) {
                stack.push(ch - '0');
            }
            else {
                int b = stack.pop();
                int a = stack.pop();
                switch (ch) {
                    case '+':
                        stack.push(a + b);
                        break;

```

```
        case '-':
            stack.push(a - b);
            break;
        case '*':
            stack.push(a * b);
            break;
        case '/':
            stack.push(a / b);
            break;
    }
}

System.out.println("Result: " + stack.pop());
}
```

OUTPUT:

Enter Postfix Expression: 23*45*+9-

Result: 17